

ANÁLISE DE VIABILIDADE

Pipeline de Provas & Comentários com IA

Viabilidade · Automação · Custos · Pré-requisitos · Otimizações

Versão 1.0 · Março 2026

Sumário

Este documento analisa em profundidade o cenário proposto — conectar provas armazenadas em JSON com comentários buscados via IA de busca e depois sintetizados pelo Claude — respondendo às cinco perguntas centrais levantadas.

Seção	Página
1. Visão Geral do Cenário	3
2. Pergunta 1 — O cenário é viável?	4
3. Pergunta 2 — É necessário código para automatizar?	7
4. Pergunta 3 — Quais os custos envolvidos?	12
5. Pergunta 4 — Quais os pré-requisitos?	18
6. Pergunta 5 — Melhorias que aceleram sem comprometer a entrega	22
7. Recomendação de Ferramenta de Busca: Perplexity vs. Alternativas	27
8. Arquitetura Final Recomendada	31
9. Cronograma e Esforço Estimado	34
10. Glossário	37

Capítulo 1: Visão Geral do Cenário

Antes de responder às perguntas individuais, é importante termos uma visão clara e compartilhada do que está sendo proposto. Esse mapeamento inicial evita mal-entendidos e serve de base para todas as análises posteriores.

1.1 O Que Se Quer Fazer

O objetivo central é criar uma conexão estruturada entre dois tipos de conteúdo:

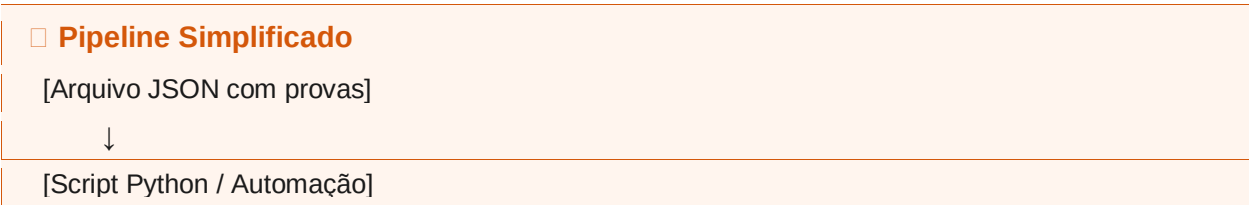
- Provas (questões de concursos, vestibulares, certificações ou similares) já coletadas por um crawler e armazenadas em um arquivo JSON organizado.
- Comentários sobre essas provas, vindos de bancas avaliadoras e de canais externos (fóruns, blogs, YouTube, redes sociais, sites especializados).

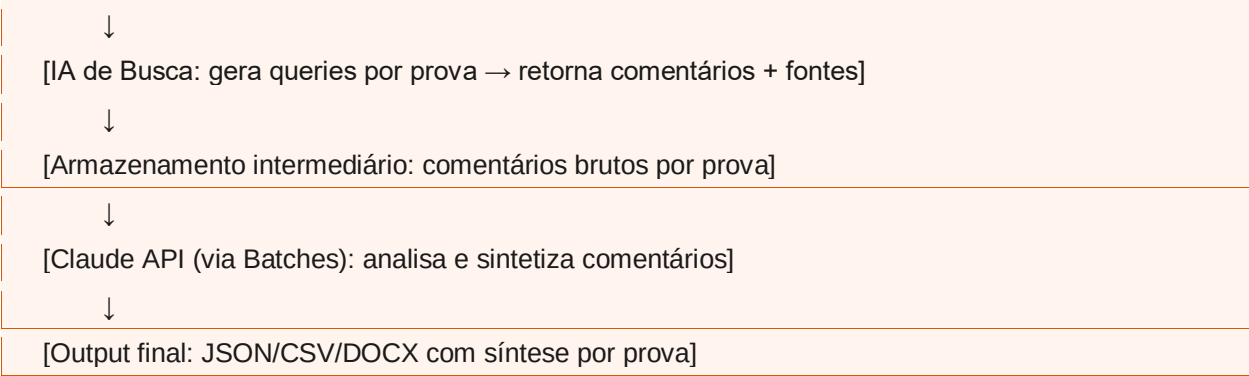
Essa conexão não é apenas armazenar os dois conjuntos de dados lado a lado — o objetivo é produzir uma síntese inteligente: associar cada prova (ou questão) aos comentários mais relevantes sobre ela e, a partir disso, gerar uma análise consolidada.

1.2 O Fluxo Proposto em Três Etapas

Etapa	Ferramenta	O que acontece
1	JSON (existente)	Provas já coletadas pelo crawler são lidas e processadas como fonte de dados estruturados.
2	IA de Busca (Perplexity ou alternativa)	Para cada prova, faz-se uma busca automatizada na web em busca de comentários, análises, gabaritos comentados e discussões da banca ou de canais relevantes.
3	Claude (Anthropic)	Com os comentários coletados na etapa 2, o Claude analisa, sintetiza e produz um output estruturado associando cada prova ao seu respectivo conteúdo de comentários.

1.3 Diagrama do Fluxo





1.4 Hipóteses de Escala

A análise de viabilidade, custos e arquitetura varia significativamente conforme o volume. Este documento considera três cenários de escala como referência:

Escala	Provas	Buscas estimadas	Chamadas ao Claude
Pequena	Até 500	500–2.500	500–1.000
Média	500–5.000	2.500–25.000	1.000–10.000
Grande	5.000–50.000	25.000–250.000	10.000–100.000

Capítulo 2: Pergunta 1 — O Cenário É Viável?

✓ Resposta direta: SIM, o cenário é viável — com ressalvas técnicas gerenciáveis.

O pipeline descrito é não apenas viável como é, de fato, um dos padrões mais estabelecidos na área de processamento de informação com IA. Trata-se de uma aplicação clássica de RAG (Retrieval-Augmented Generation): buscar dados relevantes na web e usá-los como contexto para um modelo de linguagem produzir uma análise informada.

2.1 Por Que É Viável

2.1.1 As Ferramentas Necessárias Existem e Estão Maduras

Nenhuma das três etapas do pipeline exige tecnologia experimental ou de ponta. O JSON já existe. As APIs de busca com IA (Perplexity, Tavily, Exa) são serviços comerciais estáveis com documentação extensiva. O Claude oferece API robusta com suporte a lotes (batches) que é ideal exatamente para esse tipo de processamento em massa.

2.1.2 O Padrão É Amplamente Usado em Produção

Centenas de empresas e projetos usam pipelines similares para análise de notícias, monitoramento de mercado, pesquisa acadêmica automatizada e análise de sentimento em larga escala. Não há necessidade de reinventar a roda — apenas conectar componentes já validados.

2.1.3 O Volume É Escalável

A Batches API do Claude suporta até 100.000 requisições por pacote. As APIs de busca suportam dezenas de milhares de queries por dia em planos pagos. Não há barreira técnica de escala que inviabilize o projeto.

2.2 As Ressalvas Importantes

□ Ressalva 1: Qualidade dos Comentários Depende da Popularidade da Prova

Provas muito conhecidas (ENEM, OAB, concursos federais) têm vasta cobertura online — comentários

de professores, fóruns, canais do YouTube, sites especializados. Já provas de menor porte ou mais recentes podem ter pouquíssimos comentários disponíveis na web. O pipeline entregará resultados

proporcionais à cobertura existente — e isso precisa ser levado em conta na análise de resultados.

❑ Ressalva 2: Comentários Podem Ser Conflitantes ou de Baixa Qualidade

A web contém tanto análises técnicas de alta qualidade (bancas oficiais, professores especialistas) quanto opiniões desinformadas em redes sociais. O Claude consegue filtrar e ponderar isso, mas quanto pior a qualidade das fontes encontradas, menor a qualidade da síntese final. Definir queries de busca bem estruturadas e priorizar domínios confiáveis é essencial.

❑ Ressalva 3: Conteúdo Atrás de Paywall Não Será Acessível

Alguns dos melhores comentários podem estar em plataformas pagas (cursos preparatórios, assinaturas de questões). As APIs de busca retornam apenas o conteúdo publicamente acessível. Isso é uma limitação inerente ao modelo — não uma falha técnica.

❑ Ressalva 4: Custo Pode Escalar Rapidamente

Para volumes grandes (10.000+ provas), os custos de busca e de processamento pelo Claude podem chegar a centenas de dólares por rodada. Planejamento de orçamento é essencial antes de escalar. Veja o Capítulo 4 para análise detalhada de custos.

2.3 Comparação com Alternativas

Para ser justo na avaliação, vale considerar por que esse cenário é preferível a abordagens alternativas:

Abordagem	Vantagens	Desvantagens vs. Pipeline Proposto
Busca manual (humana)	Alta qualidade de curadoria	Inviável em escala; lento e caro
Só web scraping (sem IA)	Custo zero de IA	Dados brutos e não estruturados; ruído elevado
Pipeline proposto (IA busca + IA sintetiza)	Escalável, automatizável, síntese inteligente	Custo de API; dependência de cobertura web
Banco de dados curado manualmente	Altíssima qualidade	Não escala; custo humano muito alto

2.4 Veredito

O cenário é viável, tecnicamente sólido e amplamente precedido por casos de uso similares em produção. As ressalvas apontadas não inviabilizam o projeto — elas apenas exigem atenção no

design das queries de busca, na escolha das fontes e no estabelecimento de expectativas realistas sobre a qualidade do output para provas com baixa cobertura online.

Capítulo 3: Pergunta 2 — É Necessário Código Para Automatizar?

✓ **Resposta direta: SIM, código é necessário — mas é mais simples do que parece.**

Não existe uma interface gráfica pronta que conecte automaticamente um JSON de provas, uma API de busca e o Claude numa única solução. Você precisa de um script que orquestre essas três peças. A boa notícia: esse script tem estrutura previsível, usa bibliotecas bem documentadas e não exige conhecimento avançado de programação para ser criado e mantido.

3.1 O Que o Código Precisa Fazer

1. Ler o arquivo JSON de provas e extrair os campos relevantes de cada questão (enunciado, matéria, banca, ano, etc.).
2. Para cada prova (ou grupo de provas), construir uma query de busca adequada e chamar a API da IA de busca (ex.: Perplexity Sonar ou Tavily).
3. Coletar e armazenar os comentários retornados, associando-os ao ID da prova correspondente no JSON.
4. Montar o payload de requisições para o Claude (via Batches API) — uma requisição por prova, com os comentários coletados como contexto.
5. Enviar o batch ao Claude e aguardar o processamento.
6. Coletar os resultados do batch e salvar o output final em um formato utilizável (JSON, CSV, DOCX, etc.).
7. Tratar erros: buscas sem resultado, timeouts, falhas parciais no batch.

3.2 Linguagem Recomendada

Python é a linguagem recomendada por três razões:

- O SDK oficial da Anthropic (anthropic) é mais maduro e documentado em Python.
- As principais APIs de busca (Perplexity, Tavily) têm SDKs e exemplos prioritariamente em Python.
- Processamento de JSON, manipulação de arquivos e pipelines de dados são tarefas para as quais o ecossistema Python é extremamente rico.

3.3 Estrutura do Código — Módulo por Módulo

Módulo 1: Leitura do JSON

```
import json
```

```
def carregar_provas(caminho_json: str) -> list[dict]:
    with open(caminho_json, "r", encoding="utf-8") as f:
        provas = json.load(f)
        print(f"Total de provas carregadas: {len(provas)}")
        return provas

provas = carregar_provas("provas.json")
```

Módulo 2: Construção de Queries de Busca

A qualidade das queries é o fator mais crítico do pipeline. Uma query mal formulada retorna resultados irrelevantes; uma query bem formulada encontra exatamente os comentários mais úteis.

```
def construir_query(prova: dict) -> str:
    # Adapte os campos conforme a estrutura do seu JSON
    banca = prova.get("banca", "")
    ano = prova.get("ano", "")
    disciplina = prova.get("disciplina", "")
    enunciado_resumo = prova.get("enunciado", "")[:150] # primeiros 150 chars

    query = f"comentários gabarito {banca} {ano} {disciplina} \"{enunciado_resumo}\""
    return query
```

Módulo 3: Busca de Comentários

Exemplo usando a API Sonar da Perplexity:

```
import requests
import os

def buscar_comentarios(query: str, max_resultados: int = 5) -> dict:
    headers = {
        "Authorization": f"Bearer {os.environ['PERPLEXITY_API_KEY']}",
        "Content-Type": "application/json"
    }
    payload = {
        "model": "sonar",
        "messages": [{"role": "user", "content": query}],
        "max_tokens": 1024
    }
    resp = requests.post("https://api.perplexity.ai/chat/completions",
                        headers=headers, json=payload, timeout=30)
    resp.raise_for_status()
    return resp.json()
```

Módulo 4: Montagem do Batch para o Claude

```
import anthropic

def montar_requisicoes_batch(provas_com_comentarios: list[dict]) -> list[dict]:
    requisicoes = []
    for item in provas_com_comentarios:
        prova_id = item["id"]
        comentarios = item["comentarios"]

        prompt = f"""
Você é um especialista em educação e análise de provas.

Abaixo estão comentários coletados sobre a seguinte questão de prova:
QUESTÃO: {item["enunciado"]}

COMENTÁRIOS COLETADOS:
{comentarios}

Com base nesses comentários, produza:
1. Um resumo do que especialistas dizem sobre essa questão
2. Os principais pontos de concordância e discordância
3. O nível de dificuldade percebido (fácil/médio/difícil)"""

        requisicoes.append({
            "custom_id": f"prova-{prova_id}",
            "params": {
                "model": "claude-sonnet-4-6",
                "max_tokens": 800,
                "messages": [{"role": "user", "content": prompt}]
            }
        })
    return requisicoes
```

Módulo 5: Orquestrador Principal

```
import time

def pipeline_completo(caminho_json: str):
    # 1. Carregar provas
    provas = carregar_provas(caminho_json)

    # 2. Buscar comentários para cada prova
    provas_com_comentarios = []
    for prova in provas:
        query = construir_query(prova)
        resultado_busca = buscar_comentarios(query)
        comentarios_texto = resultado_busca["choices"][0]["message"]["content"]
        prova['comentarios'] = comentarios_texto
```

```
provas_com_comentarios.append(prova)
time.sleep(0.5) # Respeitar rate limits da API de busca

# 3. Montar e enviar batch ao Claude
client = anthropic.Anthropic()
requisicoes = montar_requisicoes_batch(provas_com_comentarios)
batch = client.messages.batches.create(requests=requisicoes)
print(f"Batch enviado: {batch.id}")

# 4. Aguardar processamento
while batch.processing_status != "ended":
    time.sleep(60)
    batch = client.messages.batches.retrieve(batch.id)
    print(f"Status: {batch.processing_status}")

# 5. Coletar e salvar resultados
resultados = []
for resultado in client.messages.batches.results(batch.id):
    if resultado.result.type == "succeeded":
        resultados.append({
            "id": resultado.custom_id,
            "sintese": resultado.result.message.content[0].text
        })
with open("resultados_finais.json", "w") as f:
    json.dump(resultados, f, ensure_ascii=False, indent=2)
print("Pipeline concluído!")
```

3.4 Complexidade Real do Projeto

Componente	Dificuldade	Observação
Leitura do JSON	Baixa	Poucas linhas de Python
Integração com API de busca	Baixa/Média	SDKs disponíveis, exemplos abundantes
Construção das queries	Média	Iteração e teste são necessários para boa qualidade
Integração com Batches API	Baixa	SDK Python abstrai a complexidade
Tratamento de erros	Média	Necessário para robustez em produção
Projeto completo estimado	Média	1 a 3 dias para desenvolver; 1–2 dias para testar

Capítulo 4: Pergunta 3 — Quais os Custos Envolvidos?

□ **Resposta direta: Custos variam de ~\$10 (escala pequena) a ~\$2.500 (escala grande) por rodada.**

O pipeline tem três fontes de custo de API: a busca de comentários (Perplexity ou alternativa), o processamento pelo Claude, e opcionalmente o armazenamento de dados. Vamos analisar cada uma.

4.1 Custos da Etapa de Busca

4.1.1 Perplexity Sonar (Modelo Leve — Recomendado para Busca)

O modelo Sonar é o mais custo-eficiente da Perplexity para buscas em lote. Cada query custa US\$ 0,005 (ou seja, US\$ 5 por 1.000 buscas), e os tokens processados custam US\$ 1 por milhão (tanto input quanto output).

Escala	Buscas	Custo por busca	Total Busca
Pequena (500 provas, 3 buscas cada)	1.500	\$0,005	~\$7,50
Média (5.000 provas, 3 buscas cada)	15.000	\$0,005	~\$75,00
Grande (50.000 provas, 3 buscas cada)	150.000	\$0,005	~\$750,00

4.1.2 Tavily (Alternativa — Melhor Custo-Benefício para Alto Volume)

O Tavily tem plano gratuito de 1.000 créditos/mês (sem cartão). No modelo pago, 1 busca básica = 1 crédito = aprox. US\$ 0,003 a US\$ 0,008 dependendo do plano. Para volumes acima de 100.000 buscas/mês, a Tavily oferece descontos enterprise chegando a US\$ 0,0002/busca — muito mais barato que o Perplexity.

4.2 Custos da Etapa de Síntese com Claude

Usando a Batches API do Claude, todos os tokens são cobrados com 50% de desconto. A escolha do modelo afeta diretamente o custo e a qualidade da síntese.

Modelo	Input/MTok (batch)	Output/MTok (batch)	Ideal para	Qualidade
--------	--------------------	---------------------	------------	-----------

Claude Haiku 4.5	\$0,50	\$2,50	Volume muito alto, sínteses simples	Boa
Claude Sonnet 4.6	\$1,50	\$7,50	Equilíbrio custo/qualidade (recomendado)	Muito boa
Claude Opus 4.6	\$2,50	\$12,50	Sínteses complexas, análises profundas	Excelente

Estimativa de tokens por prova (modelo típico):

- Input: ~1.500 tokens (comentários coletados + prompt de instrução)
- Output: ~600 tokens (síntese produzida pelo Claude)

Escala	Custo Haiku (batch)	Custo Sonnet (batch)	Custo Opus (batch)
500 provas	~\$0,85	~\$3,38	~\$8,44
5.000 provas	~\$8,55	~\$33,75	~\$84,38
50.000 provas	~\$85,50	~\$337,50	~\$843,75

4.3 Custo Total Consolidado (Busca + Síntese)

Usando Perplexity Sonar + Claude Sonnet 4.6 via Batches (combinação mais recomendada):

Escala	Custo Busca	Custo Claude	TOTAL ESTIMADO
500 provas	~\$7,50	~\$3,38	~\$10,88
5.000 provas	~\$75,00	~\$33,75	~\$108,75
50.000 provas	~\$750,00	~\$337,50	~\$1.087,50

☐ **Custo por Prova Analisada**

Com a combinação Perplexity Sonar + Claude Sonnet (batch), o custo por prova fica em aproximadamente US\$ 0,02 a US\$ 0,025 — cerca de R\$ 0,12 a R\$ 0,15 por questão analisada.

Para a maioria dos projetos, esse valor é extremamente competitivo comparado ao custo de análise humana equivalente.

4.4 Custos de Infraestrutura (Opcionais)

- Armazenamento do JSON e resultados: praticamente zero (arquivos de texto são muito pequenos).
- Servidor para rodar o script: pode ser executado no seu próprio computador. Para automações recorrentes, um servidor VPS básico (US\$ 5–10/mês no DigitalOcean, AWS EC2 t3.micro, etc.) é suficiente.
- Banco de dados (se necessário): SQLite é gratuito. PostgreSQL na nuvem tem tiers gratuitos generosos (Supabase, Neon, Railway).

4.5 Custos de Desenvolvimento

Caso você não desenvolva o código internamente, o custo de desenvolvimento é o maior custo único do projeto:

Perfil do Desenvolvedor	Horas Estimadas	Custo de Mercado (BR)
Dev Python sênior (com experiência em IA)	8–16h	R\$ 800–2.400 (R\$100–150/h)
Dev Python júnior/pleno	16–32h	R\$ 960–2.240 (R\$60–70/h)
Freelancer internacional (Upwork)	10–20h	US\$ 300–800 (\$30–40/h)

Capítulo 5: Pergunta 4 — Quais os Pré-requisitos?

□ **Resposta direta: 5 pré-requisitos técnicos e 2 organizacionais — todos acessíveis.**

5.1 Pré-requisitos Técnicos

PT-1: Python 3.8+ instalado

A linguagem de programação base do pipeline. Versão mínima recomendada: 3.10 para compatibilidade com todos os SDKs utilizados.

- Instalação: python.org — gratuito para todos os sistemas operacionais.
- Verificação: execute `python --version` no terminal.

PT-2: Conta e API Key na Anthropic

8. Acesse platform.claude.com e crie uma conta.
9. Vá em API Keys e crie uma chave.
10. Instale o SDK: `pip install anthropic`
11. Configure a variável de ambiente: `export ANTHROPIC_API_KEY='sk-ant-...'`

PT-3: Conta e API Key na IA de Busca

Para o Perplexity:

12. Acesse perplexity.ai/api e crie uma conta.
13. Adicione um cartão de crédito (sem assinatura prévia — pay as you go).
14. Gere uma API Key.
15. Configure: `export PERPLEXITY_API_KEY='pplx-...'`

Para o Tavily (recomendado para testes — tem plano gratuito):

16. Acesse tavily.com e crie uma conta.
17. Não é necessário cartão de crédito para o plano gratuito (1.000 buscas/mês).
18. Instale o SDK: `pip install tavily-python`
19. Configure: `export TAVILY_API_KEY='tvly-...'`

PT-4: Estrutura do JSON de Provas Documentada

Para construir as queries de busca e os prompts do Claude, é necessário saber exatamente quais campos o JSON contém. Campos essenciais para o pipeline:

Campo	Obrigatório?	Uso no Pipeline
-------	--------------	-----------------

id / identificador único	Sim	Associar resultados às questões originais
enunciado / texto da questão	Sim	Construir query de busca e prompt do Claude
banca	Recomendado	Melhorar precisão das queries de busca
ano	Recomendado	Filtrar comentários pelo período correto
disciplina / matéria	Recomendado	Direcionar a busca para fontes especializadas
gabarito / resposta correta	Opcional	Enriquecer o prompt do Claude com a resposta esperada

PT-5: Bibliotecas Python Instaladas

```
# Instalar todas as dependências de uma vez
pip install anthropic requests tavily-python python-dotenv tqdm
```

Biblioteca	Para que serve
anthropic	SDK oficial da Anthropic para Python — acessa a Batches API
requests	Fazer chamadas HTTP à API do Perplexity (ou qualquer API REST)
tavily-python	SDK do Tavily (se usar Tavily como motor de busca)
python-dotenv	Carregar variáveis de ambiente de um arquivo .env (segurança)
tqdm	Barra de progresso para acompanhar o processamento em lote

5.2 Pré-requisitos Organizacionais

PO-1: Definição do Output Esperado

Antes de escrever uma linha de código, é preciso definir claramente o que o resultado final deve conter. Algumas perguntas a responder:

- Qual o formato do output? (JSON, CSV, DOCX, banco de dados)
- Qual nível de detalhe da síntese? (1 parágrafo, 5 tópicos, análise completa)
- O output será consumido por humanos, por outro sistema ou ambos?
- Há um template ou estrutura que a síntese deve seguir?

PO-2: Definição de Orçamento e Escala

Antes de rodar o pipeline em produção, defina:

- Quantas provas serão processadas por rodada?

- Com que frequência o pipeline será executado? (única vez, semanal, mensal)
- Qual é o orçamento máximo aceitável por rodada?

☐ **Dica: Comece Pequeno**

Recomendamos fortemente processar um lote piloto de 50–100 provas antes de rodar em escala total. Isso permite validar a qualidade das queries de busca, ajustar o prompt do Claude e confirmar os custos reais antes de comprometer o orçamento completo.

Capítulo 6: Pergunta 5 — Melhorias que Aceleraram Sem Comprometer a Entrega

Este capítulo apresenta otimizações concretas que reduzem tempo de execução, custos e complexidade operacional — sem sacrificar a qualidade do output final.

6.1 Melhoria 1: Prompt Caching no Claude

Se todas as questões de uma mesma prova compartilham o mesmo contexto geral (ex.: instruções da banca, regras de análise), use Prompt Caching para armazenar essa parte comum e não repagar pelos mesmos tokens repetidamente.

□ Economia Potencial com Cache

Tokens em cache custam 90% menos do que tokens normais na leitura (0,1x o preço base).

Se o seu system prompt tem 2.000 tokens e ele é compartilhado por 10.000 questões, você economizaria: $2.000 \text{ tokens} \times 10.000 \text{ questões} = 20\text{B tokens} \times 90\% \text{ off}$.

Com Sonnet em batch: $20\text{M} \times \$1,50 \times 90\% \text{ off} = \sim \$2,70$ de economia (vs. \$27 sem cache).

```
# Exemplo de prompt com cache_control
"params": {
  "model": "claude-sonnet-4-6",
  "max_tokens": 800,
  "system": [
    {
      "type": "text",
      "text": "Você é um especialista em educação... [instrução longa]",
      "cache_control": {"type": "ephemeral"} // Cache por 1h
    }
  ],
  "messages": [...]
```

6.2 Melhoria 2: Agrupamento por Banca/Disciplina

Em vez de processar prova por prova de forma completamente independente, agrupe questões similares para compartilhar o contexto de busca e reduzir chamadas à API de busca.

Por exemplo: em vez de fazer 3 buscas separadas para 3 questões da mesma prova do mesmo ano, faça 1 busca abrangente e distribua os resultados para as 3 questões. Isso pode reduzir o número de buscas em 50–70%.

```
# Agrupar por prova/banca/ano antes de buscar
from itertools import groupby

def agrupar_provas(provas: list) -> dict:
    grupos = {}
    for prova in provas:
        chave = f"{prova['banca']}_{prova['ano']}_{prova['disciplina']}"
        if chave not in grupos:
            grupos[chave] = []
        grupos[chave].append(prova)
    return grupos

# Uma busca por grupo, compartilhada por todas as questões do grupo
```

6.3 Melhoria 3: Processamento Paralelo das Buscas

A etapa de busca (Etapa 2) é a mais lenta porque cada chamada à API de busca leva 0,5 a 2 segundos. Se você processar sequencialmente 5.000 questões, isso pode levar 2–3 horas só nas buscas. Com paralelismo, esse tempo cai para 10–20 minutos.

```
import concurrent.futures
import time

def buscar_com_retry(prova: dict, max_tentativas: int = 3) -> dict:
    for tentativa in range(max_tentativas):
        try:
            query = construir_query(prova)
            resultado = buscar_comentarios(query)
            prova["comentarios"] = resultado
            return prova
        except Exception as e:
            if tentativa < max_tentativas - 1:
                time.sleep(2 ** tentativa) # Backoff exponencial
    return prova # Retorna sem comentários se falhar

# Executar em paralelo (máx. 10 threads simultâneas para respeitar rate limits)
with concurrent.futures.ThreadPoolExecutor(max_workers=10) as executor:
    provas_processadas = list(executor.map(buscar_com_retry, provas))
```

6.4 Melhoria 4: Escolha Dinâmica de Modelo

Nem toda questão exige a mesma profundidade de análise. Questões com poucos ou sem comentários não se beneficiam de um modelo caro como o Opus. Use lógica condicional para selecionar o modelo adequado:

```
def selecionar_modelo(quantidade_comentarios: int) -> str:
```

```
if quantidade_comentarios == 0:
    return None # Pular — sem dados para analisar
elif quantidade_comentarios < 200: # Comentários curtos
    return "claude-haiku-4-5" # Mais barato
elif quantidade_comentarios < 1000:
    return "claude-sonnet-4-6" # Balanço custo/qualidade
else: # Muitos comentários detalhados
    return "claude-opus-4-6" # Melhor análise
```

6.5 Melhoria 5: Filtragem de Domínios na Busca

Configure as APIs de busca para priorizar domínios confiáveis e relevantes. Isso melhora a qualidade dos comentários coletados e reduz ruído.

```
# Exemplo com Tavily — restringir domínios
from tavily import TavilyClient

client = TavilyClient(api_key=os.environ['TAVILY_API_KEY'])

resultado = client.search(
    query=query,
    include_domains=[
        "questoesvideos.com.br", "qconcursos.com",
        "estrategiaconcursos.com.br", "teconcursos.com.br",
        "grandeadvogado.com.br", "mege.com.br" # Adapte ao seu contexto
    ],
    search_depth="advanced",
    max_results=5
)
```

6.6 Resumo das Melhorias e Seus Impactos

Melhoria	Redução de Tempo	Redução de Custo	Complexidade Adicional
Prompt Caching	Nenhuma	10–30%	Baixa
Agrupamento de buscas	20–40%	40–60%	Baixa
Busca paralela (10 threads)	70–85%	Nenhuma	Média
Modelo dinâmico por questão	Nenhuma	15–35%	Baixa
Filtragem de domínios	5–10%	5–15%	Baixíssima
COMBINAÇÃO DE TODAS	75–90%	50–70%	Média

Capítulo 7: Recomendação de Ferramenta de Busca

Você mencionou o Perplexity como ferramenta de busca preferencial, mas pediu sugestões de alternativas melhores e mais baratas. Neste capítulo, fazemos uma análise comparativa completa dos principais candidatos.

7.1 Os Principais Candidatos

Ferramenta	Custo/1K buscas	Plano Gratuito	Otimizada p/ IA?	Melhor para
Perplexity Sonar	\$5,00	Não	Sim	Síntese rápida com citações
Perplexity Sonar Pro	\$5,00 + tokens	Não	Sim	Análises complexas e profundas
Tavily Basic	\$3,00–8,00	1.000/mês	Sim (RAG-first)	Pipelines de agentes e RAG
Exa (ex-Metaphor)	\$~5,00	Sim (limitado)	Sim (semântica)	Busca por conceito/similaridade
Brave Search API	\$3,00–5,00	2.000/mês	Parcialmente	Índice próprio, sem Google
Serper (Google SERP)	\$1,00 a \$3,00	100/mês	Não (raw SERP)	Volume alto, custo mínimo

7.2 Análise Detalhada: Perplexity

Pontos Fortes

- Integra busca + síntese numa única chamada — você recebe já um texto comentado sobre a questão.
- Cita fontes automaticamente — útil para rastreabilidade.
- Familiaridade: se já usa o produto Perplexity, a API tem comportamento similar.
- Zero Data Retention na API — dados não são usados para treinar modelos.

Pontos Fracos

- Não tem plano gratuito real para a API (\$5 de crédito no plano Pro pago é irrisório para testes).
- O resultado já vem sintetizado — você perde acesso ao conteúdo bruto de cada fonte.
- Para o pipeline, você vai sintetizar de novo com o Claude — pagar duas sínteses é redundante e mais caro.

7.3 Análise Detalhada: Tavily

Pontos Fortes

- Retorna o conteúdo bruto extraído de cada página — ideal para alimentar o Claude com material rico.
- 1.000 buscas gratuitas por mês sem cartão — ótimo para testes e projetos pequenos.
- Suporte nativo a LangChain, LlamaIndex e outros frameworks de IA — fácil integração.
- Permite filtrar por domínio, data, e profundidade — controle fino sobre as fontes.
- No alto volume, Enterprise pode chegar a US\$ 0,0002/busca — extremamente competitivo.

Pontos Fracos

- Não faz síntese automática — você recebe os trechos extraídos, não um resumo já pronto.
- Para o pipeline proposto, isso é uma vantagem (o Claude faz a síntese), não uma desvantagem.

7.4 Recomendação por Cenário

✓Para o Pipeline Proposto: Tavily é a Melhor Escolha

A lógica é simples: o Perplexity já sintetiza as informações — mas você vai pagar o Claude para sintetizar de novo. Com o Tavily, você paga apenas pela busca/extração de conteúdo bruto e deixa toda a síntese para o Claude. Isso evita redundância, dá mais controle sobre o conteúdo e é mais barato na maioria dos cenários de volume médio e alto.

Exceção: Se você quer o mínimo de código possível e está processando poucos centenas de provas com baixo orçamento de desenvolvimento, o Perplexity simplifica o pipeline.

Use Perplexity quando...	Use Tavily quando...
Volume < 500 provas e prioridade é simplicidade	Volume > 500 provas e custo importa
Você quer resultado semi-pronto com citações	Você quer conteúdo bruto para o Claude analisar
Não há recurso de desenvolvimento	Há desenvolvimento Python disponível
Teste inicial do conceito (menos código)	Produção com escalabilidade e controle de custo

Capítulo 8: Arquitetura Final Recomendada

Com base em todas as análises anteriores, esta é a arquitetura que equilibra melhor qualidade, custo, velocidade e manutenibilidade.

8.1 Stack Tecnológica Recomendada

Componente	Ferramenta Recomendada	Justificativa
Fonte de dados	JSON existente (crawler)	Já existe — reutilizar sem custo adicional
Busca de comentários	Tavily (plano pago)	Custo-benefício superior; retorna conteúdo rico para o Claude
Síntese e análise	Claude Sonnet 4.6 via Batches API	50% de desconto; qualidade excelente; suporta grandes volumes
Linguagem de programação	Python 3.10+	Ecossistema mais rico; SDKs oficiais disponíveis
Armazenamento intermediário	JSON Lines (.jsonl)	Formato nativo da Batches API; fácil de processar
Output final	JSON estruturado + CSV	Flexibilidade para consumo humano e por sistemas

8.2 Fluxo Detalhado da Arquitetura

❑ Arquitetura Completa em 7 Passos
FASE 1 — PREPARAÇÃO [1] Carregar JSON → Validar campos → Filtrar provas já processadas (se houver) [2] Agrupar por banca/ano/disciplina para otimizar buscas
FASE 2 — BUSCA (Paralela, 10 threads) [3] Para cada grupo: construir query → chamar Tavily → armazenar conteúdo bruto [4] Salvar comentários brutos em arquivo intermediário (backup de segurança)
FASE 3 — SÍNTESE (Batch) [5] Montar batch de requisições Claude: 1 requisição por questão [6] Enviar batch → polling a cada 60s → coletar resultados

FASE 4 — OUTPUT

[7] Salvar sínteses finais no JSON original (enriquecido) + exportar CSV

8.3 Estrutura de Arquivos do Projeto

```
pipeline-provas/
├── .env                      # API Keys (nunca commitar)
├── requirements.txt          # Dependências
├── config.py                 # Configurações (modelos, limites, domínios)
├── src/
│   ├── loader.py             # Leitura e validação do JSON
│   ├── query_builder.py      # Construção de queries de busca
│   ├── search_engine.py      # Integração com Tavily/Perplexity
│   ├── batch_manager.py      # Integração com Claude Batches API
│   ├── output_writer.py      # Geração do output final
│   └── main.py               # Orquestrador do pipeline
├── dados/
│   ├── entrada/provas.json   # Input do crawler
│   ├── intermediario/        # Comentários brutos por prova
│   └── saida/                 # Output final
└── logs/                     # Logs de execução e erros
```

8.4 Tratamento de Falhas e Resiliência

O pipeline em produção deve ser tolerante a falhas. As principais estratégias:

- Checkpoint: Salvar quais provas já foram processadas para poder retomar sem reprocessar.
- Retry com backoff exponencial: Para falhas temporárias nas APIs de busca.
- Separação de fases: Salvar os comentários brutos antes de enviar ao Claude — assim, se o batch falhar, você não precisa repetir as buscas.
- Processamento de erros parciais: Como visto no Capítulo de Batches API, erros individuais no batch não afetam as demais questões.

Capítulo 9: Cronograma e Esforço Estimado

Para concluir com uma visão prática, estimamos o esforço total para implementar e operar o pipeline descrito.

9.1 Fases de Desenvolvimento

#	Atividade	Esforço (Dev experiente)	Esforço (Dev iniciante)	Entregável
1	Configurar contas e APIs (Anthropic + Tavily)	2h	4h	APIs funcionando
2	Mapear estrutura do JSON e definir campos-chave	1h	2h	Documentação dos campos
3	Desenvolver módulo de busca + testar queries	4h	8h	Script de busca funcional
4	Desenvolver integração com Claude Batches	3h	6h	Script de batch funcional
5	Executar piloto com 50–100 questões	2h	3h	Resultados validados
6	Ajustar queries e prompts com base no piloto	3h	6h	Qualidade aprovada
7	Adicionar paralelismo, retry e checkpoints	4h	8h	Pipeline robusto
8	Executar em escala total e documentar	2h	3h	Output final
TOTAL		21h (~2,5 dias)	40h (~5 dias)	

9.2 Tempo de Execução do Pipeline (Após Desenvolvido)

Escala	Fase de Busca	Fase Claude (batch)	Tempo Total
500 provas	~5–10 min (paralelo)	~15–30 min	~20–40 min
5.000 provas	~30–60 min (paralelo)	~60–120 min	~90–180 min
50.000 provas	~3–5h (paralelo)	~2–4h	~5–9h

9.3 Resumo Executivo — As 5 Respostas

#	Pergunta	Resposta Resumida
1	O cenário é viável?	Sim. É um padrão consolidado (RAG) com tecnologia madura e escalável. Ressalvas: cobertura online variável e custo proporcional ao volume.
2	É necessário código?	Sim, mas é acessível. Um script Python com ~200–400 linhas resolve o problema. Pode ser desenvolvido em 2–5 dias por um desenvolvedor.
3	Quais os custos?	US\$ 0,02–0,025 por prova analisada. Escala pequena: ~US\$ 10. Média: ~US\$ 108. Grande: ~US\$ 1.087. Pode reduzir 50–70% com otimizações.
4	Quais os pré-requisitos?	Python 3.10+, API keys (Anthropic + Tavily/Perplexity), JSON estruturado com campos-chave, definição clara do output esperado.
5	Como acelerar sem comprometer?	Busca paralela (10x mais rápido), agrupamento por banca/ano (60% menos buscas), Prompt Caching (30% de economia), seleção dinâmica de modelo.

Capítulo 10: Glossário

Termo	Definição
API (Application Programming Interface)	Interface que permite que dois sistemas se comuniquem via chamadas HTTP. Todas as ferramentas deste pipeline são acessadas via API.
Batch / Lote	Conjunto de múltiplas requisições enviadas de uma vez à API do Claude para processamento paralelo assíncrono com 50% de desconto.
Crawler	Programa que navega automaticamente na web ou em sistemas para coletar dados estruturados. No contexto deste documento, é a origem do JSON de provas.
JSON (JavaScript Object Notation)	Formato de arquivo para armazenar dados estruturados em texto. É o formato de entrada do pipeline (as provas) e de saída (as sínteses).
Perplexity Sonar	API da Perplexity que combina busca na web com síntese por IA em uma única chamada. Retorna respostas já formatadas com citações de fontes.
Pipeline	Sequência automatizada de etapas de processamento de dados, onde o output de uma etapa é o input da próxima.
Prompt Caching	Recurso do Claude que armazena partes de um prompt em cache para reutilização, reduzindo o custo de tokens repetidos em até 90%.
RAG (Retrieval-Augmented Generation)	Padrão de IA onde o modelo recebe informações buscadas externamente como contexto antes de gerar sua resposta. É exatamente o que este pipeline implementa.
Rate Limit	Limite de chamadas que uma API aceita num período. Respeitar os rate limits evita erros 429 (Too Many Requests).
SDK (Software Development Kit)	Biblioteca oficial de código que simplifica o uso de uma API em uma linguagem de programação específica.
Tavily	API de busca desenvolvida especificamente para sistemas de IA. Retorna conteúdo bruto extraído das páginas (não síntese), ideal para pipelines RAG.
Token	Unidade de processamento de texto dos modelos de linguagem. Em português, aprox. 0,75 palavras por token.

Referências e Próximos Passos

- Documentação do Claude Batches API: platform.claude.com/docs/en/build-with-claude/batch-processing

- Documentação do Tavily: docs.tavily.com
- Documentação do Perplexity Sonar: docs.perplexity.ai
- SDK Python da Anthropic: github.com/anthropic/anthropic-sdk-python
- Prompt Caching: platform.claude.com/docs/en/build-with-claude/prompt-caching

As análises e estimativas de custo neste documento são baseadas em dados disponíveis em março de 2026. Preços de APIs podem variar — consulte sempre as páginas oficiais de precificação antes de tomar decisões de orçamento.